

Aim

To implement a program in C that generates Intermediate Code (Three Address Code) for simple arithmetic expressions, respecting operator precedence and generating temporary variables for subexpressions.

Algorithm with Explanation

1. **Start the program.**
2. Read the input expression (e.g., `a=b+c*d`).
3. Extract the right-hand side (RHS) part of the expression.
4. Initialize a temporary variable counter `t1`, `t2`,
5. Repeat while `*` or `/` operators exist:
 - Find the first `*` or `/`.
 - Extract the operands to the left and right of the operator.
 - Generate a temporary variable for the operation.
 - Replace the subexpression with the temporary variable.
 - Print the three-address instruction.
6. Repeat while `+` or `-` operators exist:
 - Do similar processing for `+` and `-`.
7. When only one operand remains, assign it to the original LHS variable.
8. End the program.

C Program Code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char expr[100];
int temp = 1;

void replace(char exp[], int l_start, int l_end, int r_start, int r_end, char op, char resu...
```

```

    strncat(newexp, exp, l_start);
    strcat(newexp, result);
    strcat(newexp, &exp[r_end + 1]);

    strcpy(exp, newexp);
}

void generateTAC(char exp[]) {
    char left[10], right[10], result[10];
    int i, l, r;

    while (strchr(exp, '*') || strchr(exp, '/')) {
        for (i = 0; exp[i]; i++) {
            if (exp[i] == '*' || exp[i] == '/') {
                l = i - 1;
                while (l >= 0 && isalnum(exp[l])) l--;
                l++;
                int p = 0;
                while (isalnum(exp[l + p])) left[p++] = exp[l + p];
                left[p] = '\0';

                r = i + 1;
                int q = 0;
                while (isalnum(exp[r + q])) right[q++] = exp[r + q];
                right[q] = '\0';

                sprintf(result, "t%d", temp++);
                printf("%s = %s %c %s\n", result, left, exp[i], right);
                replace(exp, l, l + p - 1, r, r + q - 1, exp[i], result);
                break;
            }
        }
    }

    while (strchr(exp, '+') || strchr(exp, '-')) {
        for (i = 0; exp[i]; i++) {
            if (exp[i] == '+' || exp[i] == '-') {
                l = i - 1;
                while (l >= 0 && isalnum(exp[l])) l--;
                l++;
                int p = 0;
                while (isalnum(exp[l + p])) left[p++] = exp[l + p];
                left[p] = '\0';

```

```

        r = i + 1;
        int q = 0;
        while (isalnum(exp[r + q])) right[q++] = exp[r + q];
        right[q] = '\0';

        sprintf(result, "t%d", temp++);
        printf("%s = %s %c %s\n", result, left, exp[i], right);
        replace(exp, l, l + p - 1, r, r + q - 1, exp[i], result);
        break;
    }
}

printf("%c = %s\n", expr[0], exp);
}

int main() {
    printf("Enter expression (e.g., a=b+c*d): ");
    scanf("%s", expr);

    char rhs[100];
    strcpy(rhs, expr + 2);
    generateTAC(rhs);

    return 0;
}

```

Sample Input and Output

Input

a=b+c*d

Output

```

t1 = c * d
t2 = b + t1
a = t2

```